

# Testing unitario con JUnit 5 y Mockito

## Qué es y qué problema resuelve

Es la práctica de escribir código que comprueba automáticamente que otro código hace lo que debe, sin correrlo a mano cada vez ni depender de que un humano recuerde probar todos los casos. Resuelve dos problemas concretos:

- **JUnit 5** da el marco para escribir y organizar esas comprobaciones (`@Test`, ciclo de vida, datos parametrizados) y ejecutarlas de forma repetible.
- **Mockito** resuelve un problema más específico: cuando la clase que quieres probar depende de otras piezas lentas o externas (una base de datos, un servidor de correo), Mockito las sustituye por dobles para poder probar la lógica propia sin arrastrar esas dependencias.

La carpeta avanza en cuatro proyectos: de "cómo se ve un test" (01), a organizarlos y agruparlos (02), a alimentarlos con muchos casos de datos (03), hasta usar dobles para aislar dependencias lentas (04).

## Dónde se ve en el código

- 01-junit-fundamentos — clases `Alumno.java` / `Boleta.java` bajo prueba; `PrimerTest.java`, `AsercionesTest.java`, `CicloDeVidaTest.java` (orden de `@BeforeAll`/`@BeforeEach`/`@AfterEach`/`@AfterAll`) y `DecimalesTest.java` (por qué comparar double necesita una tolerancia).
- 02-junit-catalogo — `Curso.java`; `CondicionesTest.java` (`@EnabledIf/condiciones`), `ExcepcionesTest.java` (`assertThrows`), `CursoNestedTest.java` (`@Nested`), `OrdenTest.java`, `TiempoTest.java` (`assertTimeout`).
- 03-junit-datos — `ValidadorCurp.java`, `Calificador.java`; `ValueSourceTest.java`, `CsvSourceTest.java` (con `src/test/resources/curps-validas.csv`), `EnumSourceTest.java`, `MethodSourceTest.java`, `RepeticionesTest.java` (`@RepeatedTest`).
- 04-mockito-dobles — el ejercicio central: `ServicioInscripcion.java` recibe por constructor a `RepositorioAlumnos`, `RepositorioCursos` y `Notificador` (nunca los crea con `new`, y por eso se pueden sustituir en un test). `ServicioInscripcionTest.java` usa `@Mock` + `@InjectMocks` de Mockito para probar la lógica de inscripción sin base de datos ni correos reales. `ConRepositorioRealTest.java` usa a propósito `RepositorioAlumnosLento.java` (300 ms simulados por consulta) para poder medir la diferencia contra los dobles.

## Qué pasa si no lo usas

- **Sin tests automatizados:** cada cambio hay que probarlo a mano, y es fácil olvidar un caso límite (cupa lleno, alumno que no existe) hasta que falla en producción.
- **Probando ServicioInscripcion contra el repositorio real en vez de con dobles:** [scripts/por-que-mockear.sh](https://github.com/robertodiaz/scripts-por-que-mockear.sh) lo mide directamente — la misma suite con el repositorio real (300 ms por consulta) tarda notablemente más que con dobles de Mockito

(microsegundos). Con 200 tests que consultan dos veces cada uno, esa diferencia son minutos de espera; y una suite que tarda minutos es una suite que la gente deja de correr antes de subir código.

- **Mockeando el Curso en vez de solo los repositorios/notificador:** [SobreMockeoTest.java](#) (sección 09 de la guía del proyecto) muestra que si mockeas la pieza que **decide** (el objeto que aplica la regla del cupo), dejas de probar esa regla
- **Sin @Mock para el Notificador real:** una suite que use el notificador de verdad mandaría correos reales cada vez que alguien corre `mvn test`, algo que no se puede deshacer.
- **Comparando double con assertEquals sin tolerancia (delta):** por cómo se representan los decimales en binario, comparaciones que "deberían" ser iguales fallan por errores de redondeo minúsculos; [DecimalesTest.java](#) en el proyecto 01 muestra por qué hace falta un delta.

## Cómo correrlo

```
cd 04-unit-testing/proyectos/04-mockito-dobles
```

```
./mvnw test
```

Para ver la comparación de velocidad entre dobles y el repositorio real:

```
./scripts/por-que-mockear.sh
```